Programmation orientée objet avec Java

Informatique — Licence 2 (L2)

Cours INF-L2-01

Institut Africain de Management (IAM) — Bamako, Mali
Bibliothèque Universitaire — Bronbah Tech

À propos de ce cours

Passer de la programmation procédurale à la programmation orientée objet avec Java : classes et objets, encapsulation, héritage, polymorphisme, classes abstraites, interfaces, gestion des exceptions et collections, jusqu'à une première application de gestion complète.

Document pédagogique original créé pour les étudiants de l'IAM : leçons détaillées, exemples commentés, travaux dirigés et pratiques, exercices corrigés, QCM d'évaluation et vidéos recommandées. Aucun contenu n'est reproduit d'ouvrages existants.

Sommaire

• Introduction à la matière
• Objectifs d'apprentissage
• Leçon 1 — De la programmation procédurale à l'objet : classes et objets
• Leçon 2 — Attributs, méthodes et constructeurs en Java
• Leçon 3 — Encapsulation : visibilité, getters et setters
• Leçon 4 — Héritage et polymorphisme
• Leçon 5 — Classes abstraites et interfaces
• Leçon 6 — Gestion des exceptions
• Leçon 7 — Les collections Java : ArrayList et HashMap
• Leçon 8 — Projet : application de gestion orientée objet
• Travaux dirigés (TD)
• Travaux pratiques (TP)
• Exercices corrigés supplémentaires
• QCM d'évaluation
• Résumé — les notions importantes
• Vidéos associées aux leçons
• Sources et références

Comment utiliser ce cours : lis chaque leçon dans l'ordre, refais les exemples à la main, tente chaque exercice AVANT de lire son corrigé, puis termine par le QCM pour vérifier ce que tu as retenu.

Introduction à la matière

En première année, tu as appris à programmer de façon procédurale : des variables d'un côté, des fonctions de l'autre, et un programme principal qui les fait travailler ensemble. Cette approche fonctionne bien pour les petits programmes, mais elle montre vite ses limites : quand les données et les traitements se multiplient, on ne sait plus quelle fonction a le droit de modifier quelle donnée, et la moindre modification devient risquée. La programmation orientée objet (POO) répond à ce problème en regroupant, dans une même entité appelée objet, les données et les comportements qui vont ensemble.

Java est un langage entièrement construit autour de ce paradigme : en Java, tout le code vit à l'intérieur de classes. Le programme source est compilé en un code intermédiaire, le bytecode, qu'une machine virtuelle (la JVM) exécute sur n'importe quel système : c'est le principe « écrire une fois, exécuter partout ». Langage des grandes applications d'entreprise, des services bancaires et du développement Android, Java est aussi un excellent langage d'apprentissage de l'objet, car il impose une structure claire et un typage rigoureux qui aident à prendre de bonnes habitudes.

Ce cours suppose que tu maîtrises les bases vues en L1 : variables, types, conditions, boucles et fonctions. Nous allons reconstruire ces acquis dans le monde objet, pas à pas : d'abord les classes et les objets, puis l'encapsulation qui protège les données, l'héritage et le polymorphisme qui évitent les répétitions, les classes abstraites et les interfaces qui organisent les grandes hiérarchies, la gestion des exceptions qui rend

les programmes robustes, et enfin les collections qui permettent de manipuler des ensembles d'objets. Le projet final assemblera toutes ces briques dans une application de gestion réaliste.

Objectifs d'apprentissage

À la fin de ce cours, tu seras capable de :

- Expliquer la différence entre programmation procédurale et orientée objet, et définir les notions de classe, d'objet, d'attribut et de méthode.
- Écrire une classe Java complète avec des attributs, des méthodes et des constructeurs, y compris des constructeurs surchargés.
- Appliquer l'encapsulation : attributs privés, getters et setters, avec des contrôles de validité dans les setters.
- Mettre en œuvre l'héritage avec extends et super, et le polymorphisme par redéfinition de méthodes (@Override).
- Choisir entre classe abstraite et interface selon le besoin, et les utiliser dans une hiérarchie de classes.
- Gérer les erreurs d'exécution avec try, catch et finally, et créer une exception personnalisée.
- Utiliser ArrayList et HashMap pour stocker, rechercher et parcourir des collections d'objets dans une application de gestion.

Leçon 1 — De la programmation procédurale à l'objet : classes et objets

Dans un programme procédural qui gère des étudiants, on déclare en général des tableaux parallèles : un tableau de noms, un tableau d'âges, un tableau de moyennes, et des fonctions séparées qui les manipulent. Le lien entre le nom d'un étudiant et sa moyenne n'existe que dans la tête du programmeur : si un tableau est trié et pas l'autre, les données deviennent fausses. L'approche objet supprime ce risque en créant un type nouveau, la classe Etudiant, qui rassemble dans un même bloc les caractéristiques (les attributs : nom, âge, moyenne) et, nous le verrons, les actions possibles sur ces données (les méthodes).

Il faut bien distinguer la classe de l'objet. La classe est un modèle, un plan de fabrication écrit une seule fois dans le code : elle décrit ce que les étudiants ont en commun. L'objet est un exemplaire concret créé à partir de ce plan pendant l'exécution : chaque étudiant réel du programme est un objet distinct, avec ses propres valeurs d'attributs. On crée un objet avec le mot-clé new, qui réserve la mémoire et appelle le constructeur de la classe. Une même classe peut donner naissance à des centaines d'objets indépendants : modifier l'âge de l'un ne change rien aux autres.

On accède aux attributs d'un objet avec la notation pointée : e1.nom lit l'attribut nom de l'objet e1. À ce stade du cours, nous déclarons les attributs sans modificateur de visibilité, donc accessibles directement ; la leçon 3 montrera pourquoi les professionnels préfèrent les rendre privés. Retiens aussi le vocabulaire : on dit qu'un

objet est une instance de sa classe, et créer un objet s'appelle instancier. Enfin, une variable comme e1 ne contient pas l'objet lui-même, mais une référence vers lui : c'est l'adresse de l'objet en mémoire, ce qui expliquera plusieurs comportements surprenants des collections et des paramètres.

L'essentiel
La classe est le modèle (attributs + méthodes) ; l'objet est un exemplaire concret créé par new. Un programme objet regroupe données et comportements au lieu de les laisser séparés comme en procédural.

Exemple — Une première classe Etudiant et deux objets

```
class Etudiant {
String nom;
int age;
double moyenne;
}

public class PremiereClasse {
public static void main(String[] args) {
Etudiant e1 = new Etudiant();
e1.nom = "Awa Diallo";
e1.age = 20;
e1.moyenne = 13.5;

Etudiant e2 = new Etudiant();
e2.nom = "Moussa Traore";
e2.age = 21;
e2.moyenne = 11.0;

System.out.println(e1.nom + " : " + e1.moyenne + "/20");
System.out.println(e2.nom + " : " + e2.moyenne + "/20");
}
}
```

À retenir

- Classe = modèle écrit une fois ; objet = instance créée par new, avec ses propres valeurs.
- La notation pointée objet.attribut permet de lire et modifier les données d'un objet précis.
- Une variable objet contient une référence (l'adresse) de l'objet, pas l'objet lui-même.

Exercice

Écris une classe Livre avec trois attributs : titre (String), auteur (String) et nbPages (int). Écris ensuite une classe de test avec une méthode main qui crée deux livres différents, remplit leurs attributs, puis affiche pour chacun une phrase du type « Java pour tous, de A. Diallo, 250 pages ».

✔ Corrigé
class Livre { String titre; String auteur; int nbPages; } — puis dans main : Livre l1 = new Livre(); l1.titre = "Java pour tous"; l1.auteur = "A. Diallo"; l1.nbPages = 250; et de même pour le second livre avec d'autres valeurs. L'affichage se fait par concaténation : System.out.println(l1.titre + ", de " + l1.auteur + ", " + l1.nbPages + " pages");. Points à vérifier dans ton code : chaque objet a été créé avec new avant tout accès à ses attributs (sinon, erreur NullPointerException à l'exécution), et les deux objets sont bien indépendants — changer l1.nbPages ne modifie pas l2.nbPages, car chaque instance possède sa propre copie des attributs.

Leçon 2 — Attributs, méthodes et constructeurs en Java

Une méthode est une fonction déclarée à l'intérieur d'une classe : elle décrit un comportement de l'objet et travaille directement sur ses attributs, sans avoir besoin de les recevoir tous en paramètres. Dans une classe CompteBancaire, la méthode deposer(montant) ajoute le montant au solde de l'objet sur lequel on l'appelle : c.deposer(20000) agit sur le compte c. La signature d'une méthode précise son type de retour (void si elle ne retourne rien), son nom et ses paramètres. Appeler une méthode sur le mauvais objet ou oublier que chaque objet a son propre état sont les deux erreurs de raisonnement les plus fréquentes chez les débutants.

Le constructeur est une méthode spéciale qui s'exécute automatiquement au moment du new : son rôle est de mettre l'objet nouveau-né dans un état cohérent, en initialisant ses attributs. Il porte exactement le nom de la classe et n'a aucun type de retour, pas même void. À l'intérieur, le mot-clé this désigne l'objet en cours de construction : this.solde = soldeInitial permet de distinguer l'attribut solde de l'objet du paramètre qui porte presque le même nom. Si tu n'écris aucun constructeur, Java en fournit un par défaut, sans paramètre ; dès que tu en écris un, ce constructeur par défaut disparaît — piège classique à connaître.

Une classe peut proposer plusieurs constructeurs : c'est la surcharge (overloading). On peut ainsi créer un compte avec seulement un titulaire (solde initial zéro) ou avec un titulaire et un solde de départ. Le compilateur choisit le bon constructeur selon le nombre et le type des arguments passés au new. La surcharge existe aussi pour les méthodes ordinaires. Bien conçus, les constructeurs rendent les objets impossibles à mettre dans un état absurde dès leur naissance : par exemple, refuser un solde initial négatif. C'est la première pierre de la fiabilité d'un programme objet.

L'essentiel
Une méthode agit sur l'état de l'objet qui la reçoit. Le constructeur, appelé par new, initialise l'objet ; il a le nom de la classe et aucun retour. Plusieurs constructeurs = surcharge, choisie selon les arguments.

Exemple — CompteBancaire : deux constructeurs et des méthodes

```
class CompteBancaire {
String titulaire;
double solde;

CompteBancaire(String titulaire) {
this.titulaire = titulaire;
this.solde = 0;
}

CompteBancaire(String titulaire, double soldeInitial) {
this.titulaire = titulaire;
this.solde = soldeInitial;
}

void deposer(double montant) {
solde = solde + montant;
}

boolean retirer(double montant) {
if (montant <= solde) {
solde = solde - montant;
return true;
}
return false;
}
}

public class TestCompte {
public static void main(String[] args) {
CompteBancaire c = new CompteBancaire("Fatou Ndiaye", 50000);
c.deposer(20000);
System.out.println("Solde : " + c.solde);
boolean ok = c.retirer(100000);
System.out.println("Retrait reussi ? " + ok);
System.out.println("Solde final : " + c.solde);
}
}
```

À retenir

• Le constructeur porte le nom de la classe, n'a pas de type de retour et s'exécute au new.

• this.attribut désigne l'attribut de l'objet courant, à distinguer d'un paramètre de même nom.

• Surcharge : plusieurs constructeurs ou méthodes de même nom, différenciés par leurs paramètres.

Exercice

Écris une classe Rectangle avec deux attributs double : longueur et largeur. Ajoute un constructeur qui reçoit les deux dimensions, une méthode perimetre() qui retourne le périmètre et une méthode aire() qui retourne la surface. Dans main, crée un rectangle de 5 sur 3 et affiche son périmètre puis son aire.

✔ Corrigé
class Rectangle { double longueur; double largeur; Rectangle(double longueur, double largeur) { this.longueur = longueur; this.largeur = largeur; } double perimetre() { return 2 * (longueur + largeur); } double aire() { return longueur * largeur; } } — dans main : Rectangle r = new Rectangle(5, 3); System.out.println(r.perimetre()); affiche 16.0, et System.out.println(r.aire()); affiche 15.0. Remarque les types : perimetre() et aire() retournent un double, donc pas de void dans leur signature, et chaque méthode se termine par un return. Le constructeur, lui, ne contient aucun return de valeur : il se contente d'initialiser les attributs avec this.

Leçon 3 — Encapsulation : visibilité, getters et setters

Dans les leçons précédentes, n'importe quel code pouvait écrire e.moyenne = 250 ou c.solde = -100000 : l'objet n'avait aucun moyen de se défendre contre des valeurs absurdes. L'encapsulation corrige ce défaut : on déclare les attributs private, ce qui les rend inaccessibles depuis l'extérieur de la classe. Le seul chemin pour lire ou modifier l'état de l'objet passe alors par des méthodes publiques prévues à cet effet. L'objet devient une boîte noire protégée : on connaît ses boutons (les méthodes), mais on ne peut plus toucher directement à ses mécanismes internes.

Les méthodes d'accès suivent une convention de nommage universelle : getNom() retourne la valeur de l'attribut nom (c'est un getter, ou accesseur), et setNom(...) la modifie (c'est un setter, ou mutateur). Pour un attribut booléen, le getter s'appelle plutôt isActif() que getActif(). L'intérêt décisif du setter est le contrôle : setMoyenne peut refuser toute valeur en dehors de l'intervalle 0 à 20, setPrix peut refuser un prix négatif. L'objet garantit ainsi que son état reste toujours valide, quoi que fasse le reste du programme — c'est un contrat de confiance entre la classe et ses utilisateurs.

Java propose quatre niveaux de visibilité : private (la classe seule), le niveau par défaut sans mot-clé (le paquetage), protected (le paquetage plus les sous-classes, leçon 4) et public (tout le monde). La règle professionnelle est simple : attributs private par défaut, méthodes public quand elles font partie du service rendu par la classe. L'encapsulation a un second bénéfice, moins visible : elle autorise à changer l'implémentation interne d'une classe (par exemple stocker une note sur 100 au lieu de 20) sans casser le code qui l'utilise, tant que les getters et setters gardent la même signature.

L'essentiel
Encapsuler = attributs private + accès contrôlé par getters et setters publics. Le setter est le garde-fou : il peut valider, convertir ou refuser une valeur, donc l'état de l'objet reste toujours cohérent.

Exemple — Un Etudiant encapsulé, avec un setter qui valide

```
class Etudiant {
private String nom;
private double moyenne;

public Etudiant(String nom, double moyenne) {
this.nom = nom;
setMoyenne(moyenne);
}

public String getNom() { return nom; }
public double getMoyenne() { return moyenne; }

public void setMoyenne(double m) {
if (m >= 0 && m <= 20) {
this.moyenne = m;
} else {
System.out.println("Moyenne refusee : " + m);
}
}

public boolean estAdmis() { return moyenne >= 10; }
}

public class TestEncapsulation {
public static void main(String[] args) {
Etudiant e = new Etudiant("Ibrahim Sow", 12.5);
System.out.println(e.getNom() + " : " + e.getMoyenne());
e.setMoyenne(25);
System.out.println("Apres tentative : " + e.getMoyenne());
System.out.println("Admis ? " + e.estAdmis());
}
}
```

À retenir

- private interdit l'accès direct aux attributs depuis les autres classes.
- Getter = lecture (getX / isX), setter = écriture contrôlée (setX).
- Un setter peut valider les données : c'est lui qui garantit la cohérence de l'objet.

Exercice

Écris une classe Produit dont le prix (double) est privé. Le constructeur reçoit le nom et le prix initial en passant par le setter. Le setter setPrix refuse tout prix strictement négatif en affichant « Prix refuse » et laisse l'ancien prix inchangé. Dans main, crée un produit à 1 500 FCFA, tente de fixer son prix à -500, puis affiche le prix final.

✔ Corrigé
class Produit { private String nom; private double prix; public Produit(String nom, double prix) { this.nom = nom; setPrix(prix); } public double getPrix() { return prix; } public void setPrix(double p) { if (p >= 0) { this.prix = p; } else { System.out.println("Prix refuse"); } } } — dans main, la tentative setPrix(-500) affiche « Prix refuse » et getPrix() retourne toujours 1500.0. Deux remarques : le constructeur appelle setPrix au lieu d'affecter directement l'attribut, pour bénéficier du même contrôle dès la création ; et si le prix initial passé au constructeur était négatif, l'attribut garderait sa valeur par défaut 0.0, car l'affectation n'a jamais lieu — un état valide, même dans le pire des cas.

Leçon 4 — Héritage et polymorphisme

Un étudiant et un enseignant d'une université partagent beaucoup de choses : un nom, un âge, une adresse e-mail. Mais l'étudiant a en plus une filière et des notes, l'enseignant une matière et un salaire. L'héritage permet d'écrire une classe Personne qui regroupe le commun, puis des classes Etudiant et Enseignant qui en héritent et ajoutent leurs particularités. En Java, on écrit class Etudiant extends Personne : la sous-classe reçoit automatiquement les attributs et méthodes de la classe parente (sauf les membres private, accessibles via les getters hérités). L'héritage traduit la relation « est un » : un étudiant EST une personne.

Le constructeur de la sous-classe doit d'abord construire la partie héritée : sa première instruction est un appel au constructeur parent, super(nom), qui doit figurer en toute première ligne. Si la classe parente n'a pas de constructeur sans paramètre, cet appel explicite est obligatoire, sinon le code ne compile pas. Une sous-classe peut aussi redéfinir une méthode du parent pour l'adapter : la méthode sePresenter() de Personne affiche un message général, celle d'Etudiant ajoute la filière. L'annotation @Override, placée avant la méthode redéfinie, demande au compilateur de vérifier que la signature correspond bien à une méthode héritée : une faute de frappe devient une erreur détectée au lieu d'un bug silencieux.

Le polymorphisme est la conséquence la plus puissante de l'héritage : une variable de type Personne peut référencer un Etudiant ou un Enseignant, et quand on appelle p.sePresenter(), c'est la méthode de l'objet réel qui s'exécute, pas celle du type déclaré. On peut donc ranger des objets de sous-classes différentes dans un même tableau de Personne et les traiter uniformément : chaque objet répond à sa façon. Java choisit la bonne méthode au moment de l'exécution, selon la classe réelle de l'objet : c'est la liaison dynamique. Grâce à elle, on peut ajouter demain une nouvelle sous-classe Secretaire sans modifier le code qui manipule des Personne.

L'essentiel
extends crée une relation « est un » : la sous-classe hérite du parent, appelle super(...) en premier et peut redéfinir ses méthodes (@Override). Le polymorphisme : une référence de type parent exécute la méthode de la sous-classe réelle de l'objet.

Exemple — Personne, Etudiant, Enseignant : un tableau polymorphe

```
class Personne {
protected String nom;
public Personne(String nom) { this.nom = nom; }
public void sePresenter() {
System.out.println("Je m'appelle " + nom);
}
}
class Etudiant extends Personne {
private String filiere;
public Etudiant(String nom, String filiere) {
super(nom);
this.filiere = filiere;
}
@Override
public void sePresenter() {
System.out.println("Je suis " + nom + ", etudiant en " + filiere);
}
}
class Enseignant extends Personne {
private String matiere;
public Enseignant(String nom, String matiere) {
super(nom);
this.matiere = matiere;
}
@Override
public void sePresenter() {
System.out.println("Je suis " + nom + ", enseignant de " + matiere);
}
}
public class TestPolymorphisme {
public static void main(String[] args) {
Personne[] tab = new Personne[3];
tab[0] = new Etudiant("Awa Diallo", "Informatique");
tab[1] = new Enseignant("Karim Bamba", "Mathematiques");
tab[2] = new Personne("Un invite");
for (Personne p : tab) {
p.sePresenter();
}
}
}
```

À retenir

• Hériter = recevoir attributs et méthodes du parent ; super(...) doit être la première instruction du constructeur fils.

• Redéfinir = réécrire une méthode héritée avec la même signature ; @Override fait vérifier par le compilateur.

• Polymorphisme : le type de la référence peut être le parent, mais c'est la méthode de l'objet réel qui s'exécute.

Exercice

Écris une classe Vehicule avec un attribut protected marque et une méthode demarrer() qui affiche « Le vehicule demarre ». Crée une sous-classe Voiture qui redéfinit demarrer() pour afficher « La voiture demarre avec la cle ». Dans main, déclare une variable de type Vehicule contenant une Voiture et appelle demarrer() : qu'est-ce qui s'affiche, et pourquoi ?

✔ Corrigé
class Vehicule { protected String marque; public Vehicule(String marque) { this.marque = marque; } public void demarrer() { System.out.println("Le vehicule demarre"); } } class Voiture extends Vehicule { public Voiture(String marque) { super(marque); } @Override public void demarrer() { System.out.println("La voiture demarre avec la cle"); } } — dans main : Vehicule v = new Voiture("Toyota"); v.demarrer(); affiche « La voiture demarre avec la cle ». Explication : le type déclaré de v est Vehicule, mais l'objet réel est une Voiture ; grâce à la liaison dynamique, Java exécute la méthode redéfinie de Voiture. C'est le cœur du polymorphisme : le même appel produit le comportement adapté à chaque objet.

Leçon 5 — Classes abstraites et interfaces

Dans la hiérarchie des formes géométriques, la classe Forme regroupe ce que Cercle, Rectangle et Triangle ont en commun, mais une « forme » toute seule n'a pas de sens concret : comment calculer l'aire d'une forme dont on ne connaît ni la nature ni les dimensions ? Java permet de déclarer une telle classe abstract : elle sert de modèle à ses sous-classes, mais on ne peut pas l'instancier — new Forme() est interdit. Elle peut contenir des méthodes abstraites, déclarées sans corps (public abstract double aire();) : chaque sous-classe concrète a alors l'obligation de les implémenter, sinon elle doit elle-même être déclarée abstraite.

L'interface pousse la logique un cran plus loin : c'est un contrat pur, une liste de méthodes sans implémentation (avant Java 8) qu'une classe s'engage à fournir avec le mot-clé implements. Une classe ne peut hériter que d'une seule classe, mais elle peut implémenter plusieurs interfaces : un oiseau peut hériter de Animal et implémenter Volant et Nageur. Une interface décrit un rôle ou une capacité — Dessinable, Comparable, Rechargeable — plutôt qu'une identité. On programme alors contre le contrat : toute classe qui implémente Dessinable peut être passée à une méthode qui attend un Dessinable, quelle que soit sa famille.

Comment choisir entre les deux ? Si les classes partagent un état et un comportement commun déjà écrit (des attributs, des méthodes concrètes), la classe abstraite factorise ce code dans un parent. Si elles partagent seulement une promesse de méthodes, sans code commun ni attributs, l'interface est l'outil adapté. Beaucoup de bibliothèques Java combinent les deux : une interface définit le contrat public, et une classe abstraite en fournit une implémentation partielle réutilisable. Dans les deux cas, le polymorphisme reste total : une variable de type Forme ou Dessinable reçoit n'importe quel objet compatible, et les méthodes abstraites ou d'interface sont résolues dynamiquement à l'exécution.

L'essentiel
Classe abstraite : modèle non instanciable, mélange de méthodes concrètes et abstraites à compléter par les sous-classes. Interface : contrat de méthodes, implémentable par n'importe quelle classe, et une classe peut en implémenter plusieurs.

Exemple — Classe abstraite Forme et interface Dessinable

```
abstract class Forme {
public abstract double aire();
public void afficherAire() {
System.out.println("Aire = " + aire());
}
}
interface Dessinable {
void dessiner();
}
class Cercle extends Forme implements Dessinable {
private double rayon;
public Cercle(double rayon) { this.rayon = rayon; }
@Override
public double aire() { return Math.PI * rayon * rayon; }
@Override
public void dessiner() { System.out.println("Je dessine un cercle"); }
}
class Rectangle extends Forme {
private double longueur;
private double largeur;
public Rectangle(double lo, double la) { longueur = lo; largeur = la; }
@Override
public double aire() { return longueur * largeur; }
}
public class TestFormes {
public static void main(String[] args) {
Forme f1 = new Cercle(2);
f1.afficherAire();
Forme f2 = new Rectangle(5, 3);
f2.afficherAire();
Dessinable d = new Cercle(1);
d.dessiner();
}
}
```

À retenir

• Une classe abstraite ne s'instancie pas ; ses méthodes abstraites sont des obligations pour les sous-classes concrètes.

• Une interface est un contrat : implements engage la classe à fournir toutes les méthodes annoncées.

• Héritage simple (une seule classe parente) mais interfaces multiples : les interfaces décrivent des capacités.

Exercice

Écris une classe abstraite Animal avec une méthode abstraite crier() et une méthode concrète dormir() qui affiche « Zzz... ». Crée deux sous-classes : Chien, dont crier() affiche « Ouaf ! », et Chat, dont crier() affiche « Miaou ! ». Dans main, range un Chien et un Chat dans un tableau de Animal et fais-les crier chacun à leur tour.

✔ Corrigé
abstract class Animal { public abstract void crier(); public void dormir() { System.out.println("Zzz..."); } } class Chien extends Animal { @Override public void crier() { System.out.println("Ouaf !"); } } class Chat extends Animal { @Override public void crier() { System.out.println("Miaou !"); } } — dans main : Animal[] animaux = { new Chien(), new Chat() }; puis une boucle for (Animal a : animaux) { a.crier(); } affiche « Ouaf ! » puis « Miaou ! ». Remarques : Chien et Chat sont obligés d'implémenter crier(), sinon le compilateur refuse de les déclarer concrets ; en revanche dormir() est héritée telle quelle, sans redéfinition. Un new Animal() dans main provoquerait une erreur de compilation, car Animal est abstraite.

Leçon 6 — Gestion des exceptions

Que se passe-t-il si un utilisateur tape « vingt » au lieu de 20, si un programme divise par zéro ou ouvre un fichier disparu ? En Java, ces incidents lèvent une exception : un objet qui décrit l'erreur et interrompt l'exécution normale. Sans traitement, l'exception remonte de méthode en méthode jusqu'à la JVM, qui affiche la pile d'appels (le message « Exception in thread main » bien connu des débutants) et arrête le programme. La gestion des exceptions permet au programme de réagir proprement : informer l'utilisateur, proposer une nouvelle saisie ou enregistrer l'incident, au lieu de planter brutalement.

Le mécanisme repose sur trois mots-clés. Le bloc try contient le code risqué ; le bloc catch, placé juste après, intercepte un type d'exception précis et contient le traitement ; le bloc finally, optionnel, s'exécute dans tous les cas, qu'une exception ait eu lieu ou non — idéal pour fermer un fichier ou une connexion. On peut enchaîner plusieurs catch du plus précis au plus général. Certaines exceptions sont dites contrôlées (checked) : le compilateur oblige à les traiter ou à les déclarer avec throws dans la signature de la méthode, comme IOException. D'autres sont non contrôlées (unchecked), héritières de RuntimeException — ArithmeticException, NullPointerException, NumberFormatException — que le compilateur n'impose pas de traiter : elles signalent en général une erreur de programmation à corriger.

Une classe peut aussi créer ses propres exceptions métier, en héritant de Exception, et les lever avec throw quand une règle est violée : par exemple lever SoldeInsuffisantException si un retrait dépasse le solde d'un compte. Lever une exception métier plutôt que retourner un code d'erreur rend les règles du domaine explicites et impossibles à ignorer silencieusement, car l'appelant est obligé de décider quoi en faire. Bonne pratique : attraper l'exception au niveau où l'on sait réagir, avec un message clair pour l'utilisateur, et ne jamais écrire un catch vide qui « avale » l'erreur sans laisser de trace.

L'essentiel
Une exception est un objet d'erreur qui interrompt le flux normal. try contient le code risqué, catch traite un type d'erreur, finally s'exécute toujours. throw lève une exception, throws la déclare ; on peut créer ses propres exceptions métier.

Exemple — Une exception personnalisée : SoldeInsuffisantException

```
class SoldeInsuffisantException extends Exception {
public SoldeInsuffisantException(String message) {
super(message);
}
}
class Compte {
private double solde;
public Compte(double solde) { this.solde = solde; }
public double getSolde() { return solde; }
public void retirer(double montant) throws SoldeInsuffisantException {
if (montant > solde) {
throw new SoldeInsuffisantException("Il manque " + (montant - solde) + " FCFA");
}
solde = solde - montant;
System.out.println("Retrait OK, solde : " + solde);
}
}
public class TestExceptions {
public static void main(String[] args) {
Compte c = new Compte(30000);
try {
c.retirer(50000);
} catch (SoldeInsuffisantException e) {
System.out.println("Erreur : " + e.getMessage());
} finally {
System.out.println("Solde actuel : " + c.getSolde());
}
}
}
```

À retenir

• try / catch / finally : le code risqué, le traitement de l'erreur, et le nettoyage toujours exécuté.

• Exceptions contrôlées (à traiter ou déclarer throws) contre non contrôlées (erreurs de programmation, famille RuntimeException).

• Créer une exception métier = hériter de Exception, la lever avec throw new, et la décrire par un message clair.

Exercice

1) Écris une méthode statique int diviser(int a, int b) qui retourne a / b, mais qui intercepte ArithmeticException pour afficher « Division par zero impossible » et retourner 0. 2) Dans main, convertis la chaîne "vingt" en entier avec Integer.parseInt,

en interceptant NumberFormatException pour afficher « Ce n'est pas un nombre ». Montre le code complet.

✔ Corrigé
public static int diviser(int a, int b) { try { return a / b; } catch (ArithmeticException e) { System.out.println("Division par zero impossible"); return 0; } } — et dans main : try { int age = Integer.parseInt("vingt"); System.out.println(age); } catch (NumberFormatException e) { System.out.println("Ce n'est pas un nombre"); }. Vérifications : diviser(10, 2) retourne 5 sans passer par le catch ; diviser(10, 0) affiche le message et retourne 0, sans planter. NumberFormatException et ArithmeticException sont des exceptions non contrôlées : le compilateur ne force pas à les attraper, mais le faire rend le programme robuste face aux saisies fantaisistes des utilisateurs.

Leçon 7 — Les collections Java : ArrayList et HashMap

Un tableau Java a une taille fixée à sa création : impossible d'y ajouter un 31e étudiant si le tableau a été dimensionné pour 30. Les collections du paquetage java.util lèvent cette limite. ArrayList est la plus utilisée : c'est une liste ordonnée qui grandit et rétrécit selon les besoins, avec des méthodes simples — add pour ajouter, get(i) pour lire l'élément d'indice i, set(i, x) pour le remplacer, remove pour supprimer, size() pour connaître le nombre d'éléments, contains pour tester la présence. On parcourt une ArrayList avec une boucle for classique sur les indices, ou avec la boucle for-each qui visite chaque élément directement, sans indice.

ArrayList est une classe générique : on précise le type des éléments entre chevrons, ArrayList<Etudiant>, et le compilateur garantit qu'aucun objet d'un autre type ne s'y glissera. Conséquence importante : les collections ne stockent que des objets, jamais des types primitifs. Pour ranger des entiers, on utilise la classe enveloppe Integer, des réels Double, des booléens Boolean ; Java convertit automatiquement dans les deux sens (autoboxing et unboxing), donc notes.add(15) fonctionne pour une ArrayList<Integer>. Déclarer le type entre chevrons évite les conversions risquées et fait détecter les erreurs de type dès la compilation au lieu de l'exécution.

HashMap répond à un autre besoin : retrouver rapidement une valeur à partir d'une clé, comme un annuaire qui associe un nom à un numéro de téléphone. On y range des paires avec put(cle, valeur), on lit avec get(cle), on teste l'existence d'une clé avec containsKey, et on supprime avec remove(cle). Les clés sont uniques : un second put avec la même clé écrase l'ancienne valeur. Attention au piège : get sur une clé absente retourne null, ce qui provoque une NullPointerException si on déboxe le résultat sans vérifier. Pour parcourir une HashMap, on itère sur keySet() (les clés) ou sur entrySet() (les paires clé-valeur). ArrayList répond à « une suite ordonnée d'éléments », HashMap à « un dictionnaire clé → valeur » : bien choisir entre les deux est une décision de conception quotidienne.

L'essentiel
ArrayList : liste dynamique et générique (add, get, remove, size, for-each), réservée aux objets — les primitifs passent par leurs classes enveloppes. HashMap : associations clé → valeur uniques (put, get, containsKey) ; get sur une clé absente retourne null.

Exemple — ArrayList d'étudiants et HashMap de notes

```
import java.util.ArrayList;
import java.util.HashMap;

public class TestCollections {
public static void main(String[] args) {
ArrayList<String> etudiants = new ArrayList<>();
etudiants.add("Awa Diallo");
etudiants.add("Moussa Traore");
etudiants.add("Fatou Ndiaye");
etudiants.remove("Moussa Traore");
System.out.println("Nombre d'etudiants : " + etudiants.size());
for (String nom : etudiants) {
System.out.println("- " + nom);
}

HashMap<String, Double> notes = new HashMap<>();
notes.put("Awa Diallo", 14.5);
notes.put("Fatou Ndiaye", 11.0);
System.out.println("Note de Fatou : " + notes.get("Fatou Ndiaye"));
for (String nom : notes.keySet()) {
System.out.println(nom + " -> " + notes.get(nom));
}
}
}
```

À retenir

- ArrayList grandit dynamiquement ; on la déclare avec le type des éléments : ArrayList<MonType>.
- Pas de primitifs dans les collections : int devient Integer, double devient Double (autoboxing automatique).
- HashMap : clés uniques, put écrase l'ancienne valeur, get retourne null si la clé est absente.

Exercice

1) Crée une ArrayList<Integer> contenant les notes 12, 9, 15 et 10, puis calcule et affiche leur moyenne. 2) Crée une HashMap<String, String> représentant un petit annuaire (nom → numéro de téléphone) avec trois entrées, affiche le numéro associé à l'un des noms, et teste avec containsKey ce qui se passe pour un nom absent.

✔ Corrigé
1) ArrayList<Integer> notes = new ArrayList<>(); notes.add(12); notes.add(9); notes.add(15); notes.add(10); int somme = 0; for (int n : notes) { somme += n; } double moyenne = somme / (double) notes.size(); — la moyenne affichée vaut 11.5. Deux détails comptent : la variable de boucle est un int (le déboxage de Integer vers int est automatique), et la division force le calcul en double par la conversion (double), sinon 46 / 4 donnerait 11 en division entière. 2) HashMap<String, String> annuaire = new HashMap<>(); annuaire.put("Awa", "70112233"); ... puis annuaire.get("Awa") retourne le numéro ; pour un nom absent, containsKey retourne false et get retournerait null — toujours vérifier avant d'utiliser le résultat d'un get incertain.

Leçon 8 — Projet : application de gestion orientée objet

Il est temps d'assembler toutes les briques du cours — classes, encapsulation, collections — dans une application complète : la gestion d'une petite bibliothèque. L'analyse commence avant le code : quels sont les objets du domaine ? Un Livre, décrit par un titre, un auteur et un état (disponible ou emprunté). Et une Bibliotheque, qui joue le rôle de gestionnaire : elle conserve l'ensemble des livres dans une ArrayList<Livre> et offre les services du métier — ajouter un livre, rechercher un livre par son titre, afficher le catalogue. Cette étape d'analyse, qui consiste à repérer les noms du problème (les futures classes) et les verbes d'action (les futures méthodes), est la compétence la plus précieuse de la conception objet.

Chaque classe respecte les principes vus dans le cours : les attributs de Livre sont private, l'état « disponible » ne change que par les méthodes emprunter() et rendre(), et la méthode toString() redéfinie donne un affichage propre de l'objet — System.out.println(livre) l'appelle automatiquement. La Bibliotheque, elle, ne connaît des livres que leur interface publique : elle appelle getTitre() et ne touche jamais leurs attributs internes. La recherche retourne le livre trouvé ou null si le titre est absent : l'appelant doit donc tester le résultat avant de l'utiliser, exactement comme avec le get d'une HashMap.

Pour faire grandir ce projet, les pistes sont nombreuses et réutilisent tout le cours : distinguer les types d'ouvrages par héritage (Roman et Manuel héritant de Livre, avec un polymorphisme sur la durée d'emprunt), lever une exception LivreIndisponibleException quand on emprunte un livre déjà sorti, ou indexer les livres par titre dans une HashMap pour une recherche instantanée. Travaille par petites étapes en recompilant après chaque ajout : c'est la méthode des professionnels. Si ton programme compile et se comporte comme prévu à chaque étape, tu tiens une application de gestion orientée objet complète, et tu maîtrises l'essentiel de ce cours.

L'essentiel
Concevoir une application objet = identifier les classes (les noms du domaine), leurs attributs privés et leurs méthodes publiques, puis les faire collaborer. La gestion d'une bibliothèque combine modèle (Livre), gestionnaire à collection (Bibliotheque) et programme principal (main).

Exemple — Mini-bibliothèque : Livre, Bibliotheque et programme principal

```
import java.util.ArrayList;
class Livre {
private String titre;
private String auteur;
private boolean disponible = true;
public Livre(String titre, String auteur) {
this.titre = titre;
this.auteur = auteur;
}
public String getTitre() { return titre; }
public boolean isDisponible() { return disponible; }
public void emprunter() { disponible = false; }
public String toString() {
String etat = disponible ? "disponible" : "emprunte";
return titre + " (" + auteur + ") - " + etat;
}
}
class Bibliotheque {
private ArrayList<Livre> livres = new ArrayList<>();
public void ajouter(Livre l) { livres.add(l); }
public Livre rechercher(String titre) {
for (Livre l : livres) {
if (l.getTitre().equals(titre)) { return l; }
}
return null;
}
public void afficherTout() {
for (Livre l : livres) { System.out.println(l); }
}
}
public class ProjetBibliotheque {
public static void main(String[] args) {
Bibliotheque b = new Bibliotheque();
b.ajouter(new Livre("Java pour tous", "A. Diallo"));
b.ajouter(new Livre("Bases de donnees", "K. Bamba"));
Livre trouve = b.rechercher("Java pour tous");
if (trouve != null) { trouve.emprunter(); }
b.afficherTout();
}
}
```

À retenir

- L'analyse objet part du domaine : les noms deviennent des classes, les verbes deviennent des méthodes.
- Redéfinir toString() donne un affichage automatique et propre de chaque objet.
- Une méthode de recherche retourne l'objet trouvé ou null : toujours tester avant usage.

Exercice

Ajoute à la classe Bibliotheque du projet une méthode int compterDisponibles() qui retourne le nombre de livres actuellement disponibles. Modifie ensuite le main pour afficher ce nombre après l'emprunt. Avec les données de l'exemple (deux livres ajoutés, un emprunté), quelle valeur doit s'afficher ?

✔ Corrigé
public int compterDisponibles() { int compteur = 0; for (Livre l : livres) { if (l.isDisponible()) { compteur++; } } return compteur; } — la méthode parcourt la collection, interroge chaque livre par son getter isDisponible() (elle ne touche jamais l'attribut privé directement) et compte ceux qui sont disponibles. Dans main, après b.afficherTout();, on ajoute System.out.println("Disponibles : " + b.compterDisponibles());. La valeur affichée est 1 : sur les deux livres du catalogue, « Java pour tous » a été emprunté, il ne reste que « Bases de donnees » disponible. Ce petit ajout illustre la force du modèle : la nouvelle fonctionnalité s'écrit en cinq lignes parce que les responsabilités étaient bien réparties entre les classes.

Travaux dirigés (TD)

TD 1 — Un panier d'achat orienté objet

Une boutique en ligne veut modéliser ses ventes. 1) Écris une classe Produit encapsulée : attributs privés nom (String), prix (double, en FCFA) et quantite (int) ; constructeur complet ; getters ; un setter setPrix qui refuse les prix négatifs ; une méthode totalLigne() qui retourne prix × quantite. 2) Écris une classe Panier qui contient une ArrayList<Produit>, une méthode ajouterProduit(Produit p), une méthode calculerTotal() qui retourne la somme des totaux de lignes, et une méthode afficherPanier() qui affiche chaque produit et le total. 3) Dans main, crée un panier avec trois produits de ton choix et affiche le panier.

✔ Corrigé du TD 1
Classe Produit : private String nom; private double prix; private int quantite; le constructeur affecte les trois attributs (en passant par setPrix pour le prix) ; public double getPrix() { return prix; } ; public void setPrix(double p) { if (p >= 0) { this.prix = p; } } ; public double totalLigne() { return prix * quantite; } ; on ajoute un toString() du type nom + " x" + quantite + " = " + totalLigne() + " FCFA" pour un affichage lisible. Classe Panier : private ArrayList<Produit> produits = new ArrayList<>(); public void ajouterProduit(Produit p) { produits.add(p); } ; public double calculerTotal() { double total = 0; for (Produit p : produits) { total += p.totalLigne(); } return total; } ; afficherPanier() parcourt la liste, affiche chaque produit (toString est appelé automatiquement), puis « TOTAL : " + calculerTotal() + " FCFA". Exemple de jeu d'essai cohérent : un produit à 2 500 FCFA en quantité 2 (5 000) et un produit à 10 000 FCFA en quantité 1 donnent un total de 15 000 FCFA. Erreurs fréquentes sanctionnées : oublier l'import java.util.ArrayList, oublier les chevrons du type générique, ou accéder à p.prix depuis Panier alors que l'attribut est privé (il faut passer par totalLigne() ou getPrix()).

TD 2 — Les employés d'une entreprise : héritage et polymorphisme

Une entreprise paie deux catégories de personnel. 1) Écris une classe Employe avec un attribut protected nom, un attribut privé salaireBase, un constructeur et une méthode calculerSalaire() qui retourne simplement le salaire de base. 2) Écris une sous-classe Commercial qui ajoute un attribut privé commission et redéfinit calculerSalaire() pour retourner salaire de base + commission. 3) Dans main, crée un tableau Employe[] contenant un Employe simple et deux Commercial, puis calcule et affiche la masse salariale totale (la somme des salaires) en une seule boucle polymorphe.

✔ Corrigé du TD 2

Classe Employe : protected String nom; private double salaireBase; public Employe(String nom, double salaireBase) { this.nom = nom; this.salaireBase = salaireBase; } public double getSalaireBase() { return salaireBase; } public double calculerSalaire() { return salaireBase; }. Classe Commercial : public Commercial(String nom, double salaireBase, double commission) { super(nom, salaireBase); this.commission = commission; } et @Override public double calculerSalaire() { return getSalaireBase() + commission; } — notez que Commercial ne peut pas lire directement salaireBase, privé dans Employe : il passe par le getter hérité, ce qui montre que l'encapsulation survit à l'héritage. Boucle de totalisation : double total = 0; for (Employe e : tab) { total += e.calculerSalaire(); } — chaque appel exécute la version de la classe réelle de l'objet, sans aucun test de type. Exemple numérique : un employé à 150 000, un commercial à 120 000 + 45 000 de commission et un commercial à 100 000 + 30 000 donnent une masse salariale de 150 000 + 165 000 + 130 000 = 445 000 FCFA.

Travaux pratiques (TP)

TP 1 — Atelier : construire et tester une classe Point

Travaux pratiques à réaliser sur machine, en binôme, en recompilant après chaque étape. Étape 1 : crée un fichier Point.java contenant une classe Point avec deux attributs privés double x et y, un constructeur qui les initialise, et les getters getX() et getY(). Étape 2 : ajoute une méthode translater(double dx, double dy) qui déplace le point (x augmente de dx, y augmente de dy). Étape 3 : ajoute une méthode distanceOrigine() qui retourne la distance du point à l'origine, calculée par Math.sqrt(x * x + y * y). Étape 4 : redéfinis toString() pour qu'elle retourne le point sous la forme "(x, y)", par exemple "(3.0, 4.0)". Étape 5 : dans une classe TestPoint avec main, crée le point (3, 4), affiche-le, affiche sa distance à l'origine, translate-le de (+1, -1), puis affiche-le à nouveau. Étape 6 (défi) : ajoute une méthode distance(Point autre) qui retourne la distance entre deux points, et vérifie-la sur les points (0, 0) et (3, 4).

✔ Attendus du TP 1
Attendus : la classe Point contient private double x; private double y; le constructeur public Point(double x, double y) { this.x = x; this.y = y; } ; translater fait this.x += dx; this.y += dy; ; distanceOrigine retourne Math.sqrt(x * x + y * y) ; toString, annotée @Override, retourne "(" + x + ", " + y + ")". Résultats attendus à l'exécution : le premier affichage donne (3.0, 4.0), la distance à l'origine vaut exactement 5.0 (le triangle 3-4-5), après translation le point s'affiche (4.0, 3.0). Pour le défi, distance(Point autre) retourne Math.sqrt((x - autre.x) * (x - autre.x) + (y - autre.y) * (y - autre.y)) — remarque que la méthode peut lire autre.x directement car autre est un objet de la même classe : private protège des autres classes, pas des objets frères. La distance entre (0, 0) et (3, 4) doit aussi donner 5.0. Critères de réussite du TP : le code compile sans erreur à chaque étape, toString est bien appelé automatiquement par println, et aucun attribut n'est accédé depuis TestPoint autrement que par les getters.

Exercices corrigés supplémentaires

Exercice 1

Surcharge de méthodes. Écris une classe Calculatrice avec trois méthodes additionner : la première prend deux int et retourne leur somme ; la deuxième prend deux double et retourne leur somme ; la troisième prend trois int et retourne leur somme. Dans main, appelle les trois versions et prédis ce qui s'affiche pour additionner(2, 3), additionner(2.5, 3.5) et additionner(1, 2, 3).

✔ Corrigé
public int additionner(int a, int b) { return a + b; } public double additionner(double a, double b) { return a + b; } public int additionner(int a, int b, int c) { return a + b + c; } — affichages : additionner(2, 3) vaut 5 (version int), additionner(2.5, 3.5) vaut 6.0 (version double), additionner(1, 2, 3) vaut 6 (version à trois paramètres). Le compilateur choisit la version uniquement d'après la liste des arguments visibles à l'appel ; le type de retour ne fait pas partie de la signature, donc deux méthodes qui ne différeraient que par leur type de retour ne peuvent pas coexister : c'est une erreur de compilation classique à connaître.

Exercice 2

Membres statiques. Écris une classe Compteur avec un attribut private static int nbObjets initialisé à 0, incrémenté dans le constructeur, et une méthode public static int getNbObjets() qui retourne sa valeur. Dans main, affiche le compteur avant toute création, crée trois objets Compteur, puis affiche à nouveau le compteur. Que vaut-il à chaque fois et pourquoi ?

✔ Corrigé
class Compteur { private static int nbObjets = 0; public Compteur() { nbObjets++; } public static int getNbObjets() { return nbObjets; } } — dans main : Compteur.getNbObjets() vaut 0 au départ, puis après trois new Compteur(), il vaut 3. Explication : un attribut static appartient à la classe entière et non à chaque objet ; il n'en existe qu'une seule copie, partagée par toutes les instances, et les trois constructeurs ont incrémenté cette même copie. On appelle la méthode statique par le nom de la classe (Compteur.getNbObjets()) sans avoir besoin d'un objet. À contraster avec un attribut d'instance, dont chaque objet possède sa propre copie, remise à zéro à chaque new.

Exercice 3

Redéfinir toString(). Écris une classe Ville avec deux attributs privés : nom (String) et habitants (int), un constructeur complet, et une redéfinition de toString() qui retourne une phrase du type « Bamako : 2 600 000 habitants ». Dans main, crée deux villes et affiche-les directement avec System.out.println(ville). Explique pourquoi l'affichage n'est pas une suite de caractères incompréhensible.

✔ Corrigé
class Ville { private String nom; private int habitants; public Ville(String nom, int habitants) { this.nom = nom; this.habitants = habitants; } @Override public String toString() { return nom + " : " + habitants + " habitants"; } } — println(ville) affiche « Bamako : 2600000 habitants ». Explication : toute classe hérite de la classe Object, dont la méthode toString() par défaut retourne un texte technique (le nom de la classe suivi d'un code). System.out.println appelle automatiquement toString() sur l'objet qu'on lui passe ; notre redéfinition remplace donc l'affichage par défaut par notre phrase. Sans cette redéfinition, le même println afficherait quelque chose comme Ville@1a2b3c4d — illisible pour un utilisateur. C'est pour cela que presque toutes les classes métier redéfinissent toString().

QCM d'évaluation

Choisis la bonne réponse pour chaque question. Les corrigés détaillés se trouvent juste après le QCM : ne les regarde qu'après avoir répondu à tout.

1. Quel mot-clé permet de créer un nouvel objet à partir d'une classe ?

   A) class

   B) new

   C) this

2. Un constructeur en Java…

   A) Ne retourne rien, pas même void, et porte exactement le nom de la classe

   B) Retourne toujours l'objet créé grâce à return

   C) Doit obligatoirement s'appeler init()

3. Déclarer un attribut private sert principalement à…

A) Le rendre accessible depuis toutes les classes

B) Accélérer l'exécution du programme

C) Interdire l'accès direct depuis l'extérieur et forcer le passage par les méthodes de la classe

4. Si on écrit class Chien extends Animal, alors…

A) Animal hérite de Chien

B) Chien hérite de Animal et reçoit ses attributs et méthodes non privés

C) Les deux classes deviennent identiques

5. À quoi sert l'annotation @Override placée avant une méthode ?

A) À créer un nouvel objet

B) À rendre la méthode statique

C) À signaler une redéfinition et à faire vérifier la signature par le compilateur

6. Que se passe-t-il si on tente d'instancier une classe abstraite avec new ?

A) Erreur de compilation : une classe abstraite ne peut pas être instanciée

B) L'objet est créé, mais ses méthodes abstraites valent null

C) Le programme plante à l'exécution avec une exception

7. Une classe Java peut…

A) Hériter de plusieurs classes en même temps

B) Implémenter plusieurs interfaces

C) Implémenter une interface seulement si elle n'hérite d'aucune classe

8. Dans un try / catch / finally, le bloc finally…

A) Ne s'exécute que si une exception est levée

B) Ne s'exécute que si tout s'est bien passé

C) S'exécute dans tous les cas, exception ou non

9. Par rapport à un tableau classique, une ArrayList…

A) A une taille qui peut grandir et rétrécir pendant l'exécution

B) Ne peut contenir que des valeurs de type int

C) Est automatiquement triée par ordre croissant

10. Dans une HashMap<String, Double>, que retourne get(cle) quand la clé est absente de la table ?

A) La valeur 0.0

B) null

C) Une exception obligatoire dans tous les cas

Corrigés du QCM

1. Réponse B — new réserve l'objet en mémoire et appelle son constructeur. class sert à déclarer une classe, this désigne l'objet courant.

2. Réponse A — Le constructeur a le nom de la classe et aucun type de retour déclaré ; c'est ce qui le distingue d'une méthode ordinaire.

3. Réponse C — C'est le mécanisme de base de l'encapsulation : l'objet contrôle les lectures et les écritures de son état.

4. Réponse B — extends se lit de la sous-classe vers la super-classe : Chien est la sous-classe, Animal la classe parente.

5. Réponse C — Si la signature ne correspond à aucune méthode héritée, le compilateur signale une erreur au lieu de créer silencieusement une nouvelle méthode.

6. Réponse A — abstract interdit l'instanciation dès la compilation ; seules les sous-classes concrètes peuvent être instanciées.

7. Réponse B — L'héritage de classes est simple en Java, mais implements accepte une liste d'interfaces séparées par des virgules.

8. Réponse C — finally sert aux nettoyages indispensables (fermer un fichier, une connexion), qui doivent avoir lieu quoi qu'il arrive.

9. Réponse A — ArrayList gère sa capacité interne automatiquement ; elle stocke des objets de n'importe quel type précisé entre chevrons.

10. Réponse B — get retourne null si la clé n'existe pas ; déboxer ce null sans le tester provoque une NullPointerException. On vérifie d'abord avec containsKey.

Résumé — les notions importantes

• Pensée objet : on modélise le problème en objets qui regroupent des données (attributs) et des comportements (méthodes) ; la classe est le modèle, l'objet est l'instance créée par new.

• Constructeurs : ils initialisent l'objet à sa naissance, portent le nom de la classe, n'ont aucun retour, peuvent être surchargés, et this désigne l'objet courant.

• Encapsulation : attributs private, accès par getters et setters ; le setter valide les valeurs et garantit que l'état de l'objet reste toujours cohérent.

• Héritage et polymorphisme : extends pour réutiliser et spécialiser, super(...) pour construire la partie héritée, @Override pour redéfinir ; à l'exécution, c'est la méthode de la classe réelle de l'objet qui est appelée.

• Classes abstraites et interfaces : la classe abstraite factorise un comportement commun sans être instanciable ; l'interface est un contrat de méthodes qu'une classe s'engage à implémenter, et on peut en implémenter plusieurs.

• Exceptions : try pour le code risqué, catch pour réagir à un type d'erreur, finally pour le nettoyage systématique ; throw lève, throws déclare, et les exceptions métier personnalisent les règles du domaine.

• Collections : ArrayList pour les listes dynamiques typées, HashMap pour les associations clé → valeur ; ce sont les briques qui, combinées aux classes métier, permettent de construire de vraies applications de gestion.

Vidéos associées aux leçons

Ces vidéos, en français, complètent les leçons. Liens vérifiés au moment de la publication de ce document.

Leçon 1 — Le langage Java : le cours complet (Java 25) · 30 chapitres, 87 exercices, de zéro à pro

Chaîne : lioncode · Langue : Français · Niveau : Débutant à avancé

Un cours complet et chapitré qui couvre presque tout ce cours : classes et objets à partir de 33 min, héritage et polymorphisme à 41 min, classes abstraites et interfaces à 45 min, exceptions à 55 min et collections à 1 h 03. Utilise les chapitres pour réviser la leçon que tu veux, dans l'ordre qui te convient.

https://www.youtube.com/watch?v=PEKoLrPbeSg

Leçon 4 — Les 4 Piliers de la POO en Java : Encapsulation, Héritage, Polymorphisme et Abstraction expliqués!

Chaîne : Solid Code App · Langue : Français · Niveau : Débutant à intermédiaire

Les quatre piliers de l'objet expliqués avec des exemples concrets en Java : protéger les données par l'encapsulation, réutiliser par l'héritage, manier les objets de façon flexible par le polymorphisme et structurer par l'abstraction. Parfait en complément des leçons 3 à 5.

https://www.youtube.com/watch?v=nns_81aOUco

Leçon 6 — Java (8/12) — La gestion des exceptions

Chaîne : Bitdyne · Langue : Français · Niveau : Intermédiaire

Un épisode entièrement consacré aux exceptions : throw et throws, try / catch / finally, exceptions contrôlées et non contrôlées, NumberFormatException et création d'une exception métier personnalisée. Le code montré est réellement compilé et exécuté à l'écran.

https://www.youtube.com/watch?v=HAMQ5PlCCBQ

Leçon 7 — Apprendre Java en 3 heures - Cours Complet Débutant (4K)

Chaîne : Comment Coder · Langue : Français · Niveau : Débutant

Un grand cours d'initiation dont la partie collections (à partir de 1 h 29) présente les listes dynamiques, ArrayList à 1 h 36 et HashMap à 1 h 41 avec des projets pratiques. La fin de la vidéo (à partir de 2 h 25) reprend aussi toute la POO : classes, constructeurs, encapsulation et héritage.

https://www.youtube.com/watch?v=KXZfh19JN-A

Sources et références

- Programme officiel Licence Informatique, Université IAM (Institut Africain de Management), Bamako — vérifier les intitulés d'UE auprès du secrétariat pédagogique.
- Documentation officielle d'Oracle — « The Java Tutorials », chapitre Learning the Java Language (docs.oracle.com/javase/tutorial/) : la référence des concepteurs du langage, en accès libre.

• Bloch, Joshua — « Effective Java » (Addison-Wesley) : les bonnes pratiques de conception objet en Java, pour aller plus loin après ce cours.

• Documentation de l'API Java (docs.oracle.com, paquetage java.util) : les fiches complètes des classes ArrayList, HashMap et de leurs méthodes.

• Les vidéos listées dans la section « Vidéos associées aux leçons » de ce document (liens vérifiés à la publication).

Fin du cours — Bronbah Tech, Bibliothèque Universitaire IAM. Bon courage pour tes révisions !